

Name: Abdul Sameer Syed Class: Soc-08 Batch: C

Subject: IDS Roll No.: 62 Exp. No.: 2

Name of the Experiment: _____

Performed on: _____

Submitted on: _____

Marks	Teacher's Signature with date
10	

Q1) Define Sequential organization. List the characteristics of linear data structures that use Sequential Organization.

→ Sequential Organization is a method of storing and managing data in which elements are arranged one after another in a contiguous memory location. Each element has a fixed position, and access is usually done in a specific order (From First to last).

Characteristics:

- 1) Contiguous memory allocation - All elements are stored in adjacent memory locations.
- 2) Fixed Size - The size of data structure is usually decided in advance.
- 3) Ordered arrangement - Elements are arranged in a linear sequence (one after another)

Incomplete for Theory ☐ Diagrams ☐ Observation Table ☐
Calculations ☐ Graphs ☐ Results ☐ Conclusion ☐
Understanding Through Q/A ☐ Late Submission ☐ Neatness ☐

Teacher's Signature

- 4) Direct access possible - Any elements can be accessed directly using its index or position.
- 5) Insertion and deletion are costly - Shifting of elements is required after insertion or deletion.
- 6) Efficient traversal - elements can be easily traversed in a single pass.

Q2. Explain the concept of an Array as an Abstract Data Type (ADT). Discuss its components and basic operations.

→ An Array ADT is a linear data structure that defines a collection of elements of the same data type, stored in contiguous memory locations. As an Abstract Data Type, it specifies what operations can be performed on the array, not how these operations are implemented internally.

Component of Array ADT

- 1) Data Elements → A finite set of elements of the same data type (e.g., integer, characters).
- 2) Index → each element is identified by an index, usually starting from 0 or 1.
- 3) Size → Total number of elements the array can store.
- 4) Base Address → The starting memory

location of array, used to calculate the address of any element.

Basic operations of Array ADT

- 1) Traversal $\rightarrow$ Accessing and processing each element of the array sequentially.
- 2) Insertion $\rightarrow$ Adding a new element at a specified position.
- 3) Deletion $\rightarrow$ Removing an element from a given position.
- 4) Merging $\rightarrow$ Combining two arrays into a single array.

Q3. Demonstrate how insertion, deletion, & search operations are performed on a ~~an~~ one dimensional array with suitable examples.

$\rightarrow$ One Dimensional Array

A one-dimensional array is a collection of elements stored in contiguous memory locations, accessed using a single index.

ex. Index: 0 1 2 3 4

Array: 10 20 30 40 50

1) Insertion in a One-Dimensional Array
Insertion means adding a new element at a specific position. To do this, elements are shifted to the right.

Ex → Insert 25 at position 2

original array: 10 20 30 40 50

steps:

1. Shift elements from the end to the right.
2. Insert the new elements at the required position.

After insertion : 10 20 25 30 40 50

2]. Deletion in a One-Dimensional Array.

Deletion means removing an element from a specific position. After deletion, elements are shifted position to the left.

ex. Delete element at position 3 (index 3)

Original array:

10 20 30 40 50

- Steps :
1. remove the element.
 2. shift remaining elements to the left.

After Deletion: 10 20 30 50

3]. Searching in a one-Dimensional Array

Searching means finding the position of a given element. The most common method is

Linear Search.

Ex. search for element 30.

Array:

10 20 30 40 50

Steps: 1). Compare each element with the key.
2). Stop when the element is found.

Result:

30 is found at index 2.

Q4. Analyse the advantages and limitations of using arrays for data storage. Compare array operations such as insertion, deletion and searching based on their efficiency.

→ Advantages OF Using Arrays for Data storage

1). Fast Access → Array allow direct access to elements using an index, which makes accessing data very fast.

2). Simple & Easy to Use → Arrays are easy to understand & implement in most programming languages

3). Efficient memory Usage → Since arrays store elements in contiguous memory locations there is not extra memory overhead.

4). Useful for fixed-size Data → Array are suitable when the number of elements is known in advance.

Limitation of Using Arrays →

1. fixed size → Once an array is created, its size cannot be changed, which may lead to memory wastage or overflow.
2. Insertion is costly → Inserting an element in the middle requires shifting elements, which is time-consuming.
3. Deletion is costly → Deleting an element also requires shifting elements to fill gap.
4. Homogeneous Data Only → Arrays store elements of the same data type, limiting flexibility.

Operation	Description	Time Complexity	Efficiency Analysis
Insertion	Adding an element	$O(n)$	Inefficient due to shifting element.
Deletion	Removing an element	$O(n)$	Inefficient due to shifting element.
searching (Linear)	searching element sequentially	$O(n)$	slow for large array
searching (Binary)	searching in sorted array	$O(\log n)$	very efficient but needs sorted array
Accessing	Access by index	$O(1)$	Highly efficient.

Q5. Design a two-dimensional array structure to store and process student academic data. Explain how the data can be accessed and manipulated efficiently. extend your discussion to the use of n-dimensional arrays in real world applications.

→ Two-Dimensional Array for Student Academic Data.

A two-dimensional (2D) array is used to store data in rows and columns, similar to a table. It is ideal for representing student academic records where each row represents a student and each column represents a student's attributes.

Design of the 2D Array Structure.

Assume we want to store:

- Student ID
- marks in three subjects : Maths, Science, English

Structure:

Students[Number of students][Number of fields]

Ex. Marks[3][4]

Student	ID	Maths	Science	English
1	101	78	85	90
2	102	88	79	84
3	103	91	92	89

This can be stored as:

Marks = {

{ 101, 78, 85, 90 },

{ 102, 88, 79, 84 },

{ 103, 91, 92, 89 }

}

Accessing Data in a Two-Dimensional Array
Elements are accessed using two indices:

Marks[row][column]

Ex : • Access Maths marks of student 2:

Marks[1][1] = 88

• Access english marks of student 3:

Marks[2][3] = 89

Manipulating Data efficiently

1. Updating Marks

Marks[0][2] = 90

2. Calculating Total Marks of a student
Add all subject marks in a row.

3. Calculating Subject Average

Add all values of a column and divide by
number of students.

Efficiency Considerations:

- Direct access (O(1)) using indices.
- Row-wise processing is cache-friendly.
- Simple looping makes computation efficient.

Extension to n-Dimensional Arrays in Real-world Applications.

An n-dimensional array extends beyond two dimensions to store complex data.

Ex. of Real-world Applications:

1. Image processing (3D Arrays)

Image[Height][Width][Color-channel]

Used to store RGB pixel values.

2. Video Processing (4D Arrays)

Video[Frame][Height][Width][Color]

3. Scientific Simulations

Used in physics and climate modeling to store multi-variable data across time and space.

4. Machine Learning (AI)

Tensors store datasets, weights, & feature maps.

5. Geographical Information System (GIS)

Store spatial data like latitude, longitude, altitude, & time.